Objetos DOM (Document Object Model)

El **Modelo de Objetos del Documento** es un conjunto de **objetos estándar para permitir el acceso a los elementos y atributos de documentos HTML y XML**. Se basa en el concepto de **nodo** (un elemento del documento) y **atributos** (cada uno de los **conjuntos par/valor** que definen el comportamiento de cada nodo). Cada nodo puede ser padre de otros nodos y tener múltiples atributos. Una **colección de nodos** es una lista de nodos (`nodeList`) y una **colección de atributos** es una lista de atributos (`attributeList`)

Supongamos que tenemos una página HTML con esta estructura:

```
<html lang="es">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<meta http-equiv="X-UA-Compatible" content="ie=edge">
<title>DOM</title>
</head>
  <body>
    <p>Esto es un párrafo</p>
      <div>
         <li>Opción 1</li>
         <li>Opción 2</li>
      </div>
  </body>
</html>
```

Si mostramos el **DOM** de la misma en la ventana de depuración del navegador veremos que adopta una **estructura de árbol jerárquica**:



DOM se puede emplear con muchos lenguajes de programación y JavaScript dispone de los objetos necesarios para poder utilizarlo. Por eso en JavaScript disponemos de **un objeto `document`** para **HTML** y **otro** para **DOM** (en realidad sólo hay un objeto que incluye ambas funcionalidades).

Vamos a ver alguno de los objetos DOM más importantes y algunos de sus métodos y funciones

Document

Como ya hemos comentado, permite acceder a la página HTML como un objeto DOM. Veamos las propiedades y métodos más interesantes

Propiedades

- `body` : Retorna una referencia al objeto `body` del documento
- `childElementCount` : Número de elementos hijos del documento
- `children` : Retorna un objeto `HTMLCollection` con todos los objetos de tipo `Element` que son hijos del documento (se omiten los elementos de texto) (`firstElementChild` y `lastElementChild` retornan respectivamente el primer y último elemento de dicha colección). Un objeto `HtmlCollection` es iterable teratiene las siguientes propiedades y métodos:
  - `length` : el número de elementos
  - `item(i)` : el elemento que está en la posición `i`
  - `namedItem(nombre)` :el elemento con el nombre indicado
- `cookie` : Retorna todas las cookies del documento
- `doctype` : Retorna el tipo de documento (`html` o `xml`) o `null` si no se ha especificado. Se retorna como un objeto `DocumentType` que dispone de la propiedad `name`
- `documentElement` : Retorna el elemento raíz del documento (el nodo HTML)
- `documentURI` : EL URI del documento (puede modificarse)
- `domain` : Retorna el nombre de dominio donde está alojada la página
- `embeds` : Retorna una lista de todos los objetos incrustados en el documento (etiqueta `<embed>`)
- `forms` : Retorna un array con todos los objetos `form` del documento
- `head` : Retorna la cabecera del documento
- `images` : Retorna una colección de todas las imágenes del documento
- `lastModified` : Retorna la fecha de última modificación del documento
- `links` : Retorna un array con todos los enlaces del documento
- `location` : Retorna la URI del documento
- `readyState` : Indica el estado del documento (`uninitialized` sin cargar, `loading` cargando, `interactive` cargado lo suficiente para que se pueda empezar a usar, `complete` completamente cargado)
- `referrer` : El URL de la página desde la que hemos llegado a este documento
- `scripts` : Una lista con todos los scripts
- `styleSheets` : Retorna una lista con todas las hojas de estilo del documento
- `title` : Pone/retorna el título del documento
- `URL` : Retorna la URL del documento

Métodos

- `createAtribute(nombre)` : Crea un atributo con el nombre especificado como un objeto `Attr`.
- `createCDATASection(texto)` : Crea una sección `CDATA`. Sólo XML
- `createComment(texto)` : Crea un comentario con el texto especificado
- `createDocumentFragment()` : Permite crear un fragmento de documento. Se crea como un nodo, al que podemos añadir otros nodos y atributos (si son del documento original se eliminan de él), modificarlos y luego insertarlo en cualquier otra parte del documento
- `createElement(nombre)` : Crea un objeto elemento con el nombre indicado
- `createEvent(nombre)` : Crea un evento
- `createTextNode(texto)` : Crea un objeto nodo de tipo texto. Normalmente los elementos HTML constan de un `ElementNode` con el elemento que es (por ejemplo `H1`) y, dentro de él, un `TextNode` con el texto asociado (lo que iría entre `<H1>` y `</H1>`)
- `getElementsByClassName(nombre)` : Retorna una lista (`HTMLCollection`) con los elementos de la clase indicada
- `getElementById(nombre)` : Retorna el elemento con el `id` especificado, `null` si no lo encuentra y `undefined` si hay varios elementos con el mismo nombre

- `getElementsByTagName(etiqueta)` : Retorna un objeto `NodeList` con todos los elementos con la `etiqueta` indicada (si se especifica la cadena vacía, retorna todos los elementos del documento). Las etiquetas son las etiquetas HTML : `H1, H2, H3, P, B`, etc. En el caso de `INPUT`, podemos emplear posteriormente la **propiedad** `type` para saber de qué tipo es
- `getElementsByName(nombre)` : Retorna una **lista iterable** de nodos (`NodeList`) con todos los elementos con la propiedad `name` indicada. Los objetos `NodeList` tienen las siguientes propiedades / métodos:
    - `length` : El número de nodos
    - `item(i)` : El nodo que está en la posición i
    - `entries()` : Un iterador con todos los pares clave / valor de la lista de nodos (suele ser la posición y el nodo)
    - `keys()` : Un iterador con todas las claves de la lista de nodos
    - `values()` : Un iterador con todos los valores de la lista de nodos
- `querySelector(selector)` : Retorna el primer elemento que coincide con el selector(es) CSS indicado(s)
- `querySelectorAll(selector)` : Retorna todos los elementos (`NodeList`) que coinciden con el selector(es) CSS indicado(s)

Selectores CSS

Para seleccionar elementos con los métodos `querySelector` y `querySelectorAll` se emplean los selectores CSS. Los selectores más importantes son:

- `selector` : Selecciona los elementos en base a su tipo (h1, p, div...)
    - `selector.clase` : selecciona los elementos del tipo y clase indicados
    - `sel1,sel2` : selecciona los elementos de tipo sel1 o sel2
- `.clase` : Selecciona los elemento que tienen la clase indicada
    - `.clase1.clase2` : selecciona elementos con ambas clases asignadas
    - `.clase1,.clase2:` selecciona los elementos con al menos una de las dos clases
    - `.clase1 .clase2` : selecciona elementos con clase2 descendientes de elementos con clase1
- `#id` : Selecciona el elemento en base a su `id`
- `*` : Selecciona todos los elementos
- `[attr]` : Selecciona los elementos que tiene el atributo indicado
    - `[attr=valor]` : Elementos con el valor indicado en el atributo
    - `[attr*=valor]` : Elementos con el valor indicado como parte del atributo
    - `[attr~=valor]` : Elementos con el valor indicado como uno de una lista de valores en el atributo separada por espacios
    - `[attr^=valor]` : Elementos cuyo atributo empieza por valor
    - `[attr$=valor]` : Elementos cuyo atributo acaba por valor
- Combinadores : Une dos selectores
    - `sel1+sel2` : Elementos donde `sel2` es el siguiente hermano de `sel1`
    - `sel1 ~sel2` : Elementos en los que `sel2` es hermano de `sel1`
    - `sel1 > sel2` : selecciona los elementos `sel2` hijos directos de `sel1`

```
Ejemplo:

    #div1 > p {

        background-color: darkblue;

    }
```

    - `sel1 sel2` : selecciona los elementos `sel2` descendientes de `sel1`
- Pseudoclases : Selecciona elementos con información que no está en el DOM. Algunas de las más habituales
    - `:active` : El elemento está activo (normalmente se aplica a botones y enlaces)
    - `:checked` : Botones de radio o checkboxes seleccionados

    - `:default` : El elemento predeterminado en un grupo de elementos de formulario
    - `:disabled / :enabled` : Elementos desactivados / activados
    - `:empty` : Elementos sin hijos
    - `:first-child / :last-child` : Seleccionado si es el primer / último hijo
    - `:first-of-type / :last-of-type` : Seleccionado si es el primer / último elemento del tipo indicado (el elemento que precede a la pseudoclase en la regla)
    - `:focus` : El elemento que recibe el enfoque
    - `:hover` : Elemento sobre el que está el ratón
    - `:in-range / :out-of-range` : El valor del elemento está / no está en el rango válido
    - `:invalid / :valid` : Elementos de un formulario que no son / son válidos
    - `:link / :visited` : Enlaces que no han sido / han sido activados
    - `:nth-child() / :nth-last-child()` : Hijos del elemento en base al valor indicado. El valor puede ser `even` (par), `odd` (impar), un `valor` que indica la posición o una fórmula para calcularlo `An+B` (`3n+2` serían los elementos `2, 5, 8, 11`...). Se busca desde el primer / último elemento.
    - `:nth-of-type() / :nth-last-of-type()` : Similar al anterior pero para los elementos de un determinado tipo
    - `:only-child` : Elementos sin hermanos
    - `:only-of-type` : Elementos sin hermanos del mismo tipo
    - `:optional / :required` : Elementos `input, select` o `textarea` sin / con el atributo `required`
    - `:read-only / :read-write` : Elementos editables / no editables
    - `:root` : Elemento raíz (HTML)
    - `:target` : Elemento cuyo `id` coincide con el del ancla interna (`#`) de un enlace
- Pseudoelementos : Van asociados a un selector (siempre devuelven una lista de elementos vacía en DOM)
    - `::after / ::before` : Último / primer hijo virtual del elemento seleccionado. Suele usarse para añadir contenido mediante la propiedad CSS content
    - `::first-letter / ::first-line` : La primera letra / línea del elemento
    - `::selection` : La zona seleccionada del elemento

Supongamos que tenemos la siguiente página web:

```
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <meta http-equiv="X-UA-Compatible" content="ie=edge">
    <title>DOM</title>
</head>
<body>
    <p name="p1">Párrafo 1</p>
    <p class="p2">Párrafo2</p>
    <div class="resaltar">
        <li>Opción 1</li>
        <li>Opción 2</li>
        <li>Opción 3</li>
        <li>Opción 4</li>
```

```
    </div>
    <div class="resaltar div2">
    <form name="f1" action="#">
        Nombre : <input type="text" id="nombre"><br>
        Contraseña : <input type="password" id="password" class="pwd"><br>
        <input type="checkbox" checked>Recordar usuario<br>
        <input type="submit" value="Enviar">
    </form>
    </div>
    <a href="#">Volver</a>
    <a href="#">Cancelar</a>
</body>
</html>
```

Veamos ejemplos de las propiedades de `document`:

```
document.head
><head>…</head>
document.body
><body>…</body>
document.childElementCount (numero de hijos del nodo(en este document)
>1
document.head.childElementCount
>4
document.body.childElementCount
>7
document.body.children
>HTMLCollection(7) [p, p.p2, div.resaltar, div.resaltar.div2, a, a, script, p1: p]
(serian del 0 al 6 los hijos y luego indica que hay p1:p)
document.body.children[3].children
>HTMLCollection [form, f1: form]
for(let s of document.body.children[3].children[0]){
    console.log(s);
}
><input type="text" id="nombre">
><input type="password" id="password" class="pwd">
><input type="checkbox" checked>
><input type="submit" value="Enviar">

(Se visualizan los elementos del formulario)
document.documentElement
><html lang="es"><head>…</head><body>…</body></html>

(es como el HTML general). No se suele usar
document.documentURI
```

>http://127.0.0.1:5500/dom.html

(indica la URL donde está desplegado)

document.forms

>HTMLCollection [form, f1: form]

document.links

>HTMLCollection(2) [a, a]

document.location

>Location {href: "http://127.0.0.1:5500/dom.html", ancestorOrigins: DOMStringList, origin: "http://127.0.0.1:5500", protocol: "http:", replace: ƒ, …}

document.readyState

>"complete"

document.referrer (Indica de donde vienes, la página anterior)

>"http://127.0.0.1:5500/dom.html"

document.scripts

>HTMLCollection [script]

document.styleSheets

(Se indica las hojas de estilo)

>StyleSheetList {length: 0}

document.title

>"DOM"

document.URL

>http://127.0.0.1:5500/dom.html

Selección de elementos

document.getElementsByName("accion")

(Buscar todos los elementos que tengan name = "acción"

>NodeList []

document.getElementsByName("p1")

>NodeList [p]

document.getElementsByClassName("resaltar")

>HTMLCollection(2) [div.resaltar, div.resaltar.div2]

document.getElementById("nombre")

><input type="text" id="nombre">

document.getElementsByTagName("li")

>HTMLCollection(4) [li, li, li, li]

document.querySelector("p")

(En este caso busca el primero que concuerde con lo escrito)

> Párrafo 1

document.querySelectorAll("p")

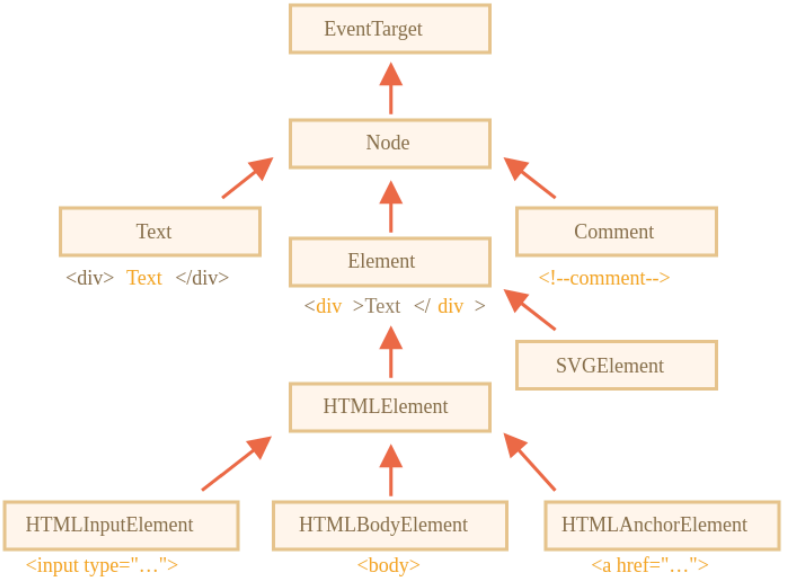
(en este caso visualiza todos los selectores, en este caso parrafo 1 y el parrafo dos que tiene la clase p2)

```
>NodeList(2) [p, p.p2]
document.querySelectorAll(".resaltar")
>NodeList(2) [div.resaltar, div.resaltar.div2]
document.querySelectorAll("#password")
(Visualiza todos los que tengan como id= password)
>NodeList [input#password.pwd]
document.querySelectorAll("p.p2")
>NodeList [p.p2]
document.querySelectorAll("p,div")
>NodeList(4) [p, p.p2, div.resaltar, div.resaltar.div2]
document.querySelectorAll("*.resaltar")
>NodeList(2) [div.resaltar, div.resaltar.div2]
document.querySelectorAll("[href]")
(saca los elementos que tengan un atributo href)
>NodeList(2) [a, a]
document.querySelectorAll("[href='#']")
(saca los elementos que tengan un atributo href con valor #)
>NodeList(2) [a, a]
document.querySelectorAll("li+li")
>NodeList(3) [li, li, li]
document.querySelectorAll("li~li")
>NodeList(3) [li, li, li]
document.querySelectorAll("p~p")
>NodeList [p.p2]
document.querySelectorAll("div>li")

(saca los elementos li que tengan como padre un div)
>NodeList(4) [li, li, li, li]
document.querySelectorAll("* input[type]")
(Te saca los elementos tipo input con atributo type)
>NodeList(4) [input#nombre, input#password.pwd, input, input]
document.querySelectorAll(":checked")
(saca los que tienen check activado)
>NodeList [input]
document.querySelectorAll(":enabled")
>NodeList(4) [input#nombre, input#password.pwd, input, input]
document.querySelectorAll("[class='resaltar'] :first-child")
>NodeList [li]
document.querySelectorAll("[class='resaltar'] :last-child")
>NodeList [li]
document.querySelectorAll(":link")
>NodeList(2) [a, a]
document.querySelectorAll(":visited")
>NodeList []
```

Node

En un documento HTML, los **nodos** son : el objeto **`document`**, los **elementos del documento** (las etiquetas), los **atributos** de estos elementos, los **textos** y los **comentarios**.



`NOTA: En el dibujo vemos que HTLMElement cuelga de Element y este a la vez de Node`

Propiedades

`Una vez recuperado un nodo, podemos recorrerlo internamente (firstChild / lastChild) o al mismo nivel (nextSibling / previousSibling) con las siguientes propiedades`

- **`childNodes`** : Retorna un **`NodeList`** con los nodos hijos
- **`firstChild / lastChild`** : Retorna el primer / último nodo
- **`nextSibling / previousSibling`** : Retorna el siguiente/anterior nodo al mismo nivel del árbol (**`null`** si estamos en el último)
- **`nodeName`** : El **nombre** del **nodo**. Si es un **elemento** se devuelve su **`tagName`**, si es un **atributo**, se devuelve su **nombre**, para el resto de elementos depende de su tipo
- **`nodeType`** : El tipo de nodo.

(para consultar los distintos tipos de nodos ver:)

https://developer.mozilla.org/es/docs/Web/API/Node/nodeType

- **`nodeValue`** : El valor del nodo
- **`outerText`** : El texto del elemento (incuido él mismo)
- **`ownerDocument`** : El objeto **`document`** en el que está el elemento
- **`parentElement`** : El nodo padre si es un elemento (**`null`** si no lo es o es el nodo raíz)
- **`parentNode`** : El nodo padre o **`null`** si no existe
- **`textContent`** : Retorna o establece el contenido de texto del nodo y sus descendientes

Métodos

- **`appendChild(node)`** : Añade un nuevo nodo al final de la lista de hijos

- `cloneNode([todo])` : Hace una copia del nodo y lo retorna como un nodo nuevo. Si se especifica todo a `true` (por defecto) se copian también sus hijos, si no sólo el nodo y sus atributos.
- `contains(nodo)` : `true` si el nodo contiene al que se pasa como parámetro
- `hasAttributes()` : Retorna un valor booleano indicando si el nodo tiene atributos
- `hasChildNodes()`: Retorna un valor booleano indicando si el nodo tiene hijos
- `insertBefore(nuevo,nodo)` : Inserta el nuevo nodo, delante del nodo indicado
- `removeChild(nodo)` : Elimina el hijo indicado
- `replaceChild(nuevo,nodo)` : Sustituye el nodo con el nuevo

Element

Representa un elemento. Como **todo elemento es un nodo**, dispone también de sus propiedades y métodos. Cada elemento puede tener como hijos otros elementos, nodos de texto, comentarios, secciones `CDATA`, instrucciones de proceso (instrucciones específicas de una aplicación que son ignoradas por los parsers de XML) y referencias a entidades (recordar que la declaración `<!ENTITY ent "texto">` permitía emplear una especie de variable en documentos XML a los que luego se accedía mediante una referencia de entidad del tipo `&ent;`)

Propiedades

- `attributes` : Retorna una colección **iterable** con los atributos del nodo como un `NamedNodeMap` con las siguientes propiedades y métodos:
  - `length` : Indica el número de elementos
  - `getNamedItem(nombre)` : Retorna el elemento en base al nombre
  - `item(pos)` : Retorna el elemento en esa posición
  - `removeNamedItem(nombre)` : Elimina el elemento indicado
  - `setNamedItem(nodo)` : Añade o sustituye el nodo a la colección
- `childElementCount` : Retorna el número de elementos hijos
- `children` : Los elementos hijos (**no incluye texto**)
- `classList` : Una lista viva **iterable** (`DOMTokenList`) con las clases CSS del elemento. Dicha lista tiene las siguientes propiedades y métodos:
  - `length` : El número de elementos
  - `value` : La representación de los elementos como una cadena
  - `item(ind)` : El elemento de la lista en la posición `ind`
  - `contains(elem)` : Retorna `true` si el elemento `elem` está en la lista
  - `add(…elem)` : Añade nuevos elementos a la lista
  - `remove(…elem)` : Elimina elementos de la lista
  - `replace(elem, elem2)` : Sustituye `elem` por `elem2`
  - `toggle(ítem)` : Si el elemento existe lo elimina y, si no existe, lo añade a la lista
  - `entries()` : Retorna un **iterador** con las **entradas** de la lista
  - `forEach(callback)` : Recorre la lista y ejecuta una función sobre cada uno de sus elementos
  - `keys()` : Retorna un **iterador** con las **claves** de la lista
  - `values()` : Retorna un **iterador** con los **valores** de la lista
- `className` : La clase del elemento
- `tagName` : El nombre de la etiqueta del elemento (en HTML en mayúsculas, en XML como esté escrito)

Métodos

- `getAttribute(att)` : Retorna el valor del atributo especificado
- `getAttributeNode(nombre)` : Retorna el atributo como un objeto `Attr`
- `getElementsByTagName(tipo)` : Retorna un array con todos los elementos del tipo indicado
- `hasAttribute(att)` : Retorna true si el elemento tiene el attributo especificado, false en caso contrario

- `removeAttribute(att)` : Elimina el atributo indicado. No retorna nada
- `removeAttrinuteNode(att)` : Igual al anterior, salvo que recibe un objeto Attr y retorna el objeto Attr eliminado
- `setAtribute(nombre,valor)` : Modifica el atributo indicado asignándole el valor
- `setAttributeNode(att)` : Añade el objeto `Attr` pasado como parámetro como un nuevo atributo del elemento (o sustituye el que hubiera si ya existe)

```
document.querySelector(".resaltar")
> <div class="resaltar">…</div>
document.querySelector(".resaltar").classList
> DOMTokenList ["resaltar", value: "resaltar"]
document.querySelector(".resaltar").children
> HTMLCollection(4) [li, li, li, li]

document.querySelector("#nombre")

> <input type="text" id="nombre">

document.querySelector("#nombre").setAttribute("disabled", true)
document.querySelector("#nombre").removeAttribute("disabled")
```

HTMLElement

Representa un elemento en el documento HTML

Propiedades

- `accessKey` : Retorna la tecla de acceso que permite acceder a ese elemento desde el teclado (en Chrome, IE y Opera `ALT+tecla`, en FireFox `ALT+SHIFT+tecla` y en Safari `Shift+Esc + tecla`)
- `dataset` : Permite acceder a **atributos definidos por el usuario** (ver más adelante)
- `draggable` : Indica / establece si el elemento puede ser arrastrado
- `hidden` : Indica / establece si el elemento está oculto
- `style` : Un objeto `CSSStyleDeclaration` representando el atributo `style` del elemento. Dicho objeto tiene las siguientes propiedades y métodos:
  - `cssText` : Permite especificar directamente el estilo como un texto
  - `length` : Número de propiedades definidas
  - `getPropertyValue(prop) / setPoperty(prop,...valores)` : Permite recuperar / establecer una propiedad CSS
  - `item(ind)` : Retorna la propiedad de la posición indicada
  - `removeProperty(propiedad)` : Elimina la propiedad

> NOTA: Las propiedades CSS se especifican como en CSS pero en **camel case y sin guiones**, es decir, `background-color` sería `backgroundColor`.

> NOTA: Los estilos también se pueden **acceder directamente** a través de la propiedad `style` (`elemento.style.estilo`)

- `tabIndex` : La posición del elemento en el orden de tabulación
- `title` : El mensaje popup que aparece al situarnos en el elemento

Métodos

- `blur()` : Elimina el enfoque del elemento
- `click()` : Simula un click en el elemento
- `focus()` : Hace que el elemento reciba el enfoque

Attr

Representa un atributo de un nodo

Propiedades

- `name` : El nombre del atributo
- `value` : El valor del atributo

Acceso a los Elementos HTML desde JavaScript

Como ya hemos visto, podemos localizar un elemento específico de un documento HTML si sabemos su id mediante el método `getElementById()`. Si queremos acceder a todos los elementos de un determinado tipo, entonces podemos emplear el método `getElementsByTagName()`. Si lo que queremos son todos los elementos de una determinada clase podemos emplea `getElementsByClassName()` y, si queremos acceder a los elementos en base a su propiedad `name` podemos emplear `getElementsByName()`.

Para esas búsquedas y otras más complejas mediante los selectores CSS podemos usar `querySelector()` y `querySelectorAll()`.

NOTA: `getElementsByTagName(), getElementsByClassName(), querySelector()` y `querySelectorAll()` pueden emplearse también desde los elementos simplificando el proceso de búsqueda.

Una vez situados en un elemento podemos acceder a sus hijos mediante la propiedad `childNodes` (incluye nodos de texto y comentarios) o mediante `children` si sólo queremos objetos de tipo `Element` (lo más habitual).

Desde un nodo podemos acceder a sus nodos hermanos mediante las propiedades `previousSibling` y `nextSibling` y a sus elementos hermanos mediante `previousElementSibling` y `nextElementSiblig.` Además podemos acceder al elemento padre mediante la propiedad `parentNode` (nodo) o `parentElement` (elemento). El elemento padre de toda la jerarquía es el elemento `document.`

Si lo que queremos es acceder a los atributos del elemento, entonces tenemos el elemento `attributes` que, por ser un objeto `Element,` tiene las mismas propiedades y métodos que acabamos de ver.

Se puede acceder al contenido de cada nodo, mediante la propiedad `nodeValue` (`null` para elementos que no tiene valor (por ejemplo un div) y el contenido para nodos de texto y comentarios), a su nombre mediante `nodeName,` a su tipo DOM mediante `nodeType` y a su etiqueta mediante `tagName`. Por último, si es un elemento HTML podemos acceder a su información mediante `innerHTML`, `outerHTML`, `innerText` y `outerText` (y, en el caso de controles de formulario, mediante la propiedad `value`).

Nota: Recordar que en HTML los nodos incluyen los comentarios y los textos mientras que los elementos no

Nota: A los atributos podemos acceder a través de la propiedad `attributes` o directamente a través de su nombre.

```
let el = document.body.children
for(e of el) {
    console.log(`${e.nodeName} (${e.nodeType}) : ${e.value}`);
}
>P (1) : undefined (p en consola los saca dos veces)
>DIV (1) : undefined (div en consola lo saca dos veces
>A (1) : undefined (en consola lo saca dos veces)
el = document.forms[0].children
for(e of el) {
    console.log(`${e.nodeName} (${e.nodeType}) : ${e.value}`);
}
```

```
>INPUT (1) : Juan
>BR (1) : undefined
>INPUT (1) : passwor
>BR (1) : undefined
>INPUT (1) : on
>BR (1) : undefined
>INPUT (1) : Enviar
```

Atributos de Datos

A la hora de pasar información entre los elementos HTML y el Javascript es bastante habitual emplear **atributos personalizados** (dado que los elementos son objetos y los atributos propiedades esto es perfectamente válido). Para **evitar interferencias con posibles nombres de atributos**, en ES6 se incluyeron los **atributos de datos**. Dichos atributos se **especifican** en el HTML como `data-atributo` y se **accede** a ellos desde Javascript con `elem.dataset.atributo`.

Supongamos que tenemos:

```
<form name="f1" action="#" data-guardar="true">

(Con estos dataset guardamos información a nivel del DOM y no se visualizan. Siempre se guarda en formato cadena, luego si se necesita se transforma)
```

Podemos acceder a dicha propiedad con :

```
document.querySelector("[name=f1]").dataset.guardar
>"true"
```

Y podríamos modificar su valor con:

```
document.querySelector("[name=f1]").dataset.guardar = false;
```

- Editar
-